DEVELOPMENT DOCUMENT

1. Project Identity

Project Name: AI-Powered Integrated Bid Compliance Verification Platform for GeM Procurement

SIH Problem Statement ID: 26100

Development Goal:
Build a secure, modular, production-oriented web application that automates tender requirement extraction, bidder document processing, compliance verification, human review, and final report generation.

---

2. Development Philosophy

The application must be developed using:

```
Clean Architecture
+
Modular Backend
+
Secure API Design
+
AI + Deterministic Rules
+
Human-in-the-Loop
+
Evidence-Based Decisions
+
Complete Auditability
```

Do NOT build the entire application in one pass or one large file.

Development must proceed incrementally.

---

3. Recommended Technology Stack

Frontend

- Next.js / React
- TypeScript
- Tailwind CSS
- Component-based UI

- TanStack Query or equivalent server-state solution
- Form validation library

Backend

- Python 3.12+
- FastAPI
- SQLAlchemy 2.x
- Pydantic v2
- Alembic
- PostgreSQL

AI

Create an AI provider abstraction.

Possible providers:

- OpenAI-compatible provider
- Other LLM provider
- Local model provider in future

Never hard-code one AI vendor into the business logic.

OCR

Use an OCR abstraction.

Possible implementations:

- Tesseract
- Cloud OCR
- Future provider

Storage

Use a storage abstraction.

Development:

- Local secure storage

Production:

- S3-compatible/object storage

Background Processing

Development:

- FastAPI background tasks where appropriate

Production:

- Celery/RQ/Arq or equivalent queue + workers

Deployment

- Docker
- Docker Compose for local development
- PostgreSQL container
- Production-ready environment configuration

---

4. Repository Structure

Use the following high-level structure:

```
bidverify/
│
├── frontend/
│
├── backend/
│
├── worker/
│
├── storage/
│
├── docs/
│
├── tests/
│
├── scripts/
│
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
└── LICENSE
```

Backend:

```
backend/
└── app/
    ├── main.py
    │
    ├── api/
    │   └── v1/
    │
    ├── core/
    │
    ├── models/
    │
    ├── schemas/
    │
    ├── repositories/
    │
    ├── services/
    │
    ├── ai/
    │
    ├── ocr/
    │
    ├── compliance/
    │
    ├── verification/
    │
    ├── review/
    │
    ├── reporting/
    │
    ├── audit/
    │
    └── utils/
```

---

5. Development Phases

Development must follow these phases:

```
Phase 0 → Project Setup
Phase 1 → Database
Phase 2 → Authentication + RBAC
Phase 3 → Tender Management
Phase 4 → Bid Management
Phase 5 → Document Management
Phase 6 → OCR + Document AI
Phase 7 → Requirement Extraction
Phase 8 → Compliance Engine
```

```
Phase 9 → Verification
Phase 10 → Human Review
Phase 11 → Reports
Phase 12 → Dashboards
Phase 13 → Security Hardening
Phase 14 → Testing
Phase 15 → Deployment
```

Do not start advanced AI features before the basic data model and authorization system are stable.

---

6. PHASE 0 — Project Setup

Tasks

- Create repository
- Create frontend
- Create backend
- Configure Python environment
- Configure Node environment
- Create `.env.example`
- Configure Git
- Configure linting
- Configure formatting
- Configure testing
- Configure Docker
- Create initial README

Backend Dependencies

Minimum categories:

```
fastapi
uvicorn
sqlalchemy
psycopg
alembic
pydantic
pydantic-settings
python-multipart
passlib/appropriate password hashing library
JWT/auth library
```

Add OCR/AI/reporting dependencies only when those modules are implemented.

---

7. PHASE 1 — Database

Use PostgreSQL.

Implement SQLAlchemy models and Alembic migrations.

Core entities:

```
User
Role
Permission
UserRole

Tender
TenderDocument
TenderRequirement

Bid
BidDocument

DocumentExtraction
ExtractedEntity
Evidence

ComplianceResult
ComplianceRule

VerificationRecord

ReviewTask
ReviewDecision

Report
Notification

AuditLog
```

---

8. Database Relationships

Core relationships:

```
User
 ├── Bids
 └── ReviewTasks
```

```
Tender
 ├── TenderDocuments
 ├── TenderRequirements
 └── Bids

Bid
 └── BidDocuments

BidDocument
 └── DocumentExtraction

DocumentExtraction
 └── Evidence

TenderRequirement
 └── ComplianceResult

ComplianceResult
 ├── Evidence
 ├── VerificationRecord
 └── ReviewTask

ReviewTask
 └── ReviewDecision
```

Use foreign keys.

Use indexes for frequently searched fields.

Use unique constraints where logically required.

---

9. PHASE 2 — Authentication

Implement:

```
POST /api/v1/auth/login
POST /api/v1/auth/logout
GET  /api/v1/auth/me
```

Authentication requirements:

- Secure password hashing
- Token/session expiration
- Authentication middleware
- User status checking
- Secure error responses

Never store plaintext passwords.

Never store secrets in source code.

---

10. PHASE 2 — RBAC

Implement roles:

```
ADMIN
PROCUREMENT_OFFICER
BIDDER
REVIEWER
AUDITOR
```

Create permission checks.

Example:

```
ADMIN:
users.read
users.write
audit.read
settings.write

OFFICER:
tenders.read
tenders.write
bids.read
compliance.read
reports.read

BIDDER:
tenders.read
bids.create
bids.read_own
documents.create_own

REVIEWER:
reviews.read_assigned
reviews.write
compliance.read

AUDITOR:
audit.read
reports.read
verification.read
```

Authorization must be checked server-side.

---

11. Resource-Level Authorization

Do not only check roles.

Check resource ownership/access.

Example:

```
Bidder A
    ↓
GET /api/v1/bids/100
    ↓
Does bid 100 belong to Bidder A?
    ↓
YES → Allow
NO  → Reject
```

Apply the same principle to:

- Bids
- Documents
- Tender information
- Reports
- Review tasks

---

12. PHASE 3 — Tender Management

Implement:

```
POST   /api/v1/tenders
GET    /api/v1/tenders
GET    /api/v1/tenders/{id}
PUT    /api/v1/tenders/{id}
DELETE /api/v1/tenders/{id}
```

Tender fields:

```
id
tender_number
title
description
```

```
organization
category
submission_deadline
status
created_by
created_at
updated_at
```

Implement valid tender state transitions.

---

13. Tender Document Processing

Endpoint:

```
POST /api/v1/tenders/{id}/documents
```

Flow:

```
Upload
 ↓
Validate
 ↓
Store
 ↓
Create processing job
 ↓
Extract text/OCR
 ↓
AI requirement extraction
 ↓
Validate extracted structure
 ↓
Save requirements
```

---

14. PHASE 4 — Bid Management

Implement:

```
POST /api/v1/bids
GET /api/v1/bids
GET /api/v1/bids/{id}
```

```
PUT /api/v1/bids/{id}
POST /api/v1/bids/{id}/submit
```

Validate:

- Tender exists
- Tender is open
- Bidder is authorized
- Required fields are present
- Submission deadline rules

---

15. PHASE 5 — Document Management

Implement secure document service.

Required capabilities:

- Upload
- Metadata storage
- Secure retrieval
- Preview where supported
- Processing status
- Retry processing
- Authorized download

Document statuses:

```
UPLOADED
VALIDATING
PROCESSING
PROCESSED
FAILED
REVIEW_REQUIRED
```

---

16. Secure Upload Pipeline

Implement:

```
Client
 ↓
Authentication
 ↓
Authorization
 ↓
```

```
File size validation
 ↓
MIME validation
 ↓
Extension validation
 ↓
Filename sanitization
 ↓
Hash generation
 ↓
Secure storage
 ↓
Processing queue
```

Never use user-provided filenames directly as filesystem paths.

Prevent:

- Path traversal
- Arbitrary file execution
- Oversized uploads
- Unsupported formats

---

17. PHASE 6 — OCR

Create:

```
OCRProvider
```

Interface:

```
extract_text(document) -> OCRResult
```

Implement at least one provider.

OCRResult:

```
text
pages
confidence
status
error
```

For digital PDFs:

```
PDF text extraction
```

For scanned documents:

```
OCR
```

Automatically choose the appropriate processing path.

---

18. PHASE 6 — Document Classification

Classify documents into:

```
GST_CERTIFICATE
PAN
REGISTRATION
FINANCIAL
EXPERIENCE
TECHNICAL
AUTHORIZATION
MSME
OTHER
```

Classification must have confidence.

Low-confidence classification:

```
REVIEW_REQUIRED
```

---

19. PHASE 7 — Tender Requirement Extraction

Create:

```
RequirementExtractor
```

Input:

```
Tender Text
```

Output:

```
Requirement[]
```

Each requirement:

```
id
category
requirement_text
mandatory
field
operator
threshold
unit
source_document
source_page
confidence
```

AI output must be validated using Pydantic schemas.

---

20. Requirement Extraction Rules

The AI should identify:

- Financial thresholds
- Technical thresholds
- Experience
- Certifications
- Mandatory documents
- Registration
- Eligibility
- Dates
- Quantities
- Other conditions

Always preserve original text.

Never replace the original tender requirement with an AI paraphrase.

---

21. PHASE 8 — Entity Extraction

Create:

```
EntityExtractor
```

Extract:

```
organization_name
gst_number
pan_number
registration_number
certificate_number
issue_date
expiry_date
financial_amount
address
authorized_person
technical_values
```

Store:

```
value
normalized_value
confidence
document_id
page_number
evidence_text
```

---

22. Data Normalization

Create a dedicated normalization layer.

Normalize:

- Names
- Dates
- Currency
- Numbers
- Whitespace
- Case
- Units

Example:

```
₹50 Lakh
₹5,000,000
50,00,000
```

may be normalized to a common numeric representation while retaining the original value.

---

23. Evidence Model

Evidence is mandatory for important compliance decisions.

Each evidence object should contain:

```
document_id
page_number
text
field
value
confidence
source_type
created_at
```

Example:

```
Requirement:
Minimum turnover ₹50 lakh

Evidence:
Document: FinancialStatement.pdf
Page: 4
Value: ₹72 lakh
Confidence: 0.94
```

---

24. PHASE 8 — Compliance Engine

Create:

```
ComplianceEngine
```

Input:

```
TenderRequirement
+
Evidence
+
VerificationResult
```

Output:

```
ComplianceResult
```

Possible statuses:

```
COMPLIANT
NON_COMPLIANT
NEEDS_REVIEW
NOT_APPLICABLE
```

---

25. Rule Engine

Implement deterministic rules first.

Examples:

```
turnover >= required_turnover
```

```
experience_years >= required_years
```

```
certificate_expiry >= required_date
```

```
mandatory_document exists
```

```
technical_value >= required_value
```

Rules must be unit-testable.

---

26. AI Compliance Assistance

AI should handle cases such as:

- Semantic matching
- Natural-language interpretation
- Ambiguous requirements
- Complex evidence relationships

AI must return structured output.

Example:

```
decision
reason
evidence_ids
confidence
requires_review
```

AI must never invent evidence.

---

27. Decision Aggregation

Use this priority:

```
Strong deterministic evidence
        ↓
Deterministic rule
        ↓
Verification
        ↓
AI semantic analysis
        ↓
Human review if uncertain
```

If the system cannot confidently establish compliance:

```
NEEDS_REVIEW
```

Do not force a binary result.

---

28. PHASE 9 — Cross-Document Verification

Implement:

```
CrossDocumentVerifier
```

Compare:

```
GST name
PAN name
Registration name
Certificate details
Dates
```

```
Addresses
Financial values
Technical values
```

Flow:

```
Extract
 ↓
Normalize
 ↓
Compare
 ↓
Detect inconsistency
 ↓
Create verification flag
```

Do not label a mismatch as fraud automatically.

---

29. Certificate Validation

Implement generic validation utilities:

```
validate_issue_date()
validate_expiry_date()
validate_against_submission_date()
```

Example:

```
expiry_date < submission_date
```

→ flag according to configured tender rule.

---

30. PHASE 9 — Verification Service

Create:

```
VerificationProvider
```

Interface:

```
verify(entity) -> VerificationResult
```

Providers:

```
MockVerificationProvider
ExternalVerificationProvider
FutureProvider
```

The business logic must depend on the interface, not the implementation.

---

31. Mock Verification

Create realistic synthetic responses.

Example:

```
Input:
GSTIN = DEMO123

Output:
status = VERIFIED
provider = MOCK
reference = MOCK-001
timestamp = ...
```

UI must clearly display:

MOCK VERIFICATION

Never represent mock data as real government verification.

---

32. PHASE 10 — Human Review

Create:

```
ReviewTaskService
```

Automatically create review tasks when:

- Confidence is below threshold
- Evidence conflicts
- OCR is uncertain

- Verification fails
- Requirement is ambiguous
- Required evidence is unavailable

---

33. Review Workflow

```
NEEDS_REVIEW
      ↓
Review Task Created
      ↓
Reviewer Opens Case
      ↓
Views Requirement
      ↓
Views Evidence
      ↓
Views Verification
      ↓
Makes Decision
      ↓
Adds Reason
      ↓
Final Decision
```

---

34. Reviewer Override

If reviewer changes an automated result:

Store:

```
original_decision
new_decision
reviewer_id
reason
timestamp
```

Never overwrite the original decision without preserving history.

---

35. PHASE 11 — Report Generation

Create:

```
ReportService
```

Generate:

- Compliance summary
- Requirement results
- Evidence
- Verification results
- Reviewer decisions
- Exceptions
- Audit references

Output:

```
PDF
```

Reports must be generated from stored system data rather than regenerated from AI prompts.

---

36. PHASE 12 — Frontend Development

Build UI after core backend APIs are stable.

Pages:

```
/login

/dashboard

/tenders
/tenders/new
/tenders/:id
/tenders/:id/requirements

/bids
/bids/:id
/bids/:id/documents
/bids/:id/compliance

/documents/:id

/reviews
/reviews/:id

/reports

/audit
```

```
/users
/settings
/profile
```

---

37. Frontend Component Architecture

Create reusable components:

```
Button
Input
Select
Modal
Dialog
Card
Badge
StatusBadge
DataTable
Tabs
Breadcrumb
Toast
Alert
Progress
FileUploader
DocumentViewer
EvidenceViewer
ComplianceCard
VerificationBadge
ReviewPanel
AuditTimeline
```

Avoid duplicated UI logic.

---

38. API Client Layer

Frontend should use a dedicated API client.

Example:

```
frontend/
└── src/
    ├── api/
    ├── components/
    ├── features/
```

```
├── hooks/
├── layouts/
├── pages/
├── types/
└── utils/
```

Do not make raw API calls from every component.

---

39. Frontend Security

Never expose:

- Database credentials
- AI API keys
- Storage credentials
- Backend secrets

Frontend must never connect directly to PostgreSQL.

All sensitive operations must go through the backend.

---

40. PHASE 13 — Security Hardening

Perform security review for:

Authentication

- Password hashing
- Token expiry
- Session invalidation
- Brute-force protection

Authorization

- RBAC
- Resource-level authorization
- IDOR protection
- Privilege escalation protection

Files

- MIME validation
- Extension validation
- Size limits
- Filename sanitization
- Secure storage
- No execution

API

- Input validation
- Rate limiting
- CORS
- Secure headers
- Safe errors

Database

- Parameterized queries/ORM
- Least privilege
- Transactions
- Connection security

---

41. Secrets Management

Create:

```
.env.example
```

Example:

```
DATABASE_URL=
JWT_SECRET=
AI_API_KEY=
STORAGE_SECRET=
```

Never commit actual values.

Add:

```
.env
```

to `.gitignore`.

Use production secret management in deployment.

---

42. Audit Implementation

Create a centralized audit service:

```
AuditService.log(
    actor,
    action,
    resource,
    result,
    metadata
)
```

Log:

```
LOGIN
LOGOUT
TENDER_CREATED
TENDER_UPDATED
DOCUMENT_UPLOADED
DOCUMENT_ACCESSED
AI_PROCESSING
VERIFICATION_REQUESTED
COMPLIANCE_DECISION
REVIEW_DECISION
REPORT_GENERATED
ROLE_CHANGED
```

Do not log secrets or unnecessary sensitive document content.

---

43. Error Handling

Use centralized exceptions.

Example:

```
AuthenticationError
AuthorizationError
ValidationError
NotFoundError
FileValidationError
OCRProcessingError
AIProcessingError
VerificationError
DatabaseError
```

API responses should follow a consistent structure.

Example:

```
{
  "error": {
    "code": "DOCUMENT_PROCESSING_FAILED",
    "message": "The document could not be processed."
  }
}
```

Do not expose stack traces in production.

---

44. Background Processing

The following operations should be asynchronous:

```
OCR
AI extraction
Document classification
Compliance analysis
External verification
PDF report generation
```

Flow:

```
API
 ↓
Create Job
 ↓
Queue
 ↓
Worker
 ↓
Process
 ↓
Database Update
 ↓
Frontend Poll/WebSocket
```

For MVP, begin with a simple implementation and make the architecture replaceable with a queue.

---

45. Processing Status API

Provide status endpoints.

Example:

```
GET /api/v1/documents/{id}/status
GET /api/v1/bids/{id}/processing-status
```

Return:

```
status
progress
current_stage
error
updated_at
```

---

46. Logging

Use structured logs.

Include:

```
timestamp
request_id
operation
user_id where appropriate
status
duration
error_code
```

Never log:

```
passwords
API keys
tokens
full confidential documents
```

---

47. Testing Strategy

Create:

```
tests/
├── unit/
├── integration/
├── api/
```

```
├── security/
└── e2e/
```

---

48. Unit Tests

Test:

- Requirement parsing
- Number normalization
- Date validation
- Rule engine
- Compliance evaluation
- Permission functions
- Entity matching
- Verification adapters

---

49. Integration Tests

Test:

```
Authentication
Tender creation
Tender upload
Requirement extraction
Bid creation
Document upload
OCR
Compliance
Verification
Review
Report
Audit
```

---

50. Security Tests

Mandatory tests:

IDOR

Bidder A cannot access Bidder B's bid.

Document Access

Bidder A cannot access Bidder B's documents.

Role Escalation

Bidder cannot call Admin endpoints.

File Upload

Unsupported/malicious files are rejected.

Token

Expired/invalid tokens are rejected.

Injection

Inputs cannot alter database queries.

---

51. End-to-End Test Scenario

Use synthetic data.

```
Admin
 ↓
Create Officer
 ↓
Create Bidder
 ↓
Officer creates Tender
 ↓
Upload Tender
 ↓
Requirements extracted
 ↓
Bidder submits Bid
 ↓
Bidder uploads documents
 ↓
OCR/Extraction
 ↓
Compliance Engine
 ↓
Verification
 ↓
Some cases become NEEDS_REVIEW
 ↓
```

```
Reviewer reviews
 ↓
Final decision
 ↓
Generate PDF
 ↓
Audit trail updated
```

This flow must work before calling the MVP complete.

---

52. Demo Dataset

Create synthetic documents/data for:

Compliant

All requirements satisfied.

Missing

Mandatory document absent.

Expired

Certificate expired.

Financial Failure

Turnover below required threshold.

Mismatch

Organization names differ.

Uncertain

Low OCR/AI confidence.

The demo dataset must not contain real confidential procurement data.

---

53. API Documentation

FastAPI OpenAPI documentation must be enabled for development.

Document:

- Endpoint
- Request schema
- Response schema
- Authentication
- Required role
- Error responses

Keep API versioned:

```
/api/v1/
```

---

54. Database Migration Strategy

Use Alembic.

Rules:

1. Never manually modify production schema.
2. Every schema change gets a migration.
3. Migration must be reversible where practical.
4. Test migrations before deployment.

---

55. Git Workflow

Use feature branches.

Example:

```
main
develop

feature/auth
feature/tender-management
feature/document-processing
feature/compliance-engine
feature/reviewer-workflow
feature/reporting
```

Commit messages should be meaningful.

Example:

```
feat: implement tender requirement extraction
fix: prevent unauthorized bid document access
test: add compliance rule tests
```

---

56. Definition of Done

A feature is complete only when:

```
Code
+
Database
+
API
+
Validation
+
Authorization
+
Error Handling
+
Tests
+
Documentation
```

are implemented as applicable.

Do not consider a feature complete merely because its frontend screen exists.

---

57. Development Priority

Follow this priority strictly:

P0

```
Project Setup
PostgreSQL
Authentication
RBAC
Tender
Bid
Secure Documents
```

P1

```
OCR
Requirement Extraction
Entity Extraction
Compliance Engine
Evidence
Verification
Review Workflow
```

P2

```
Reports
Notifications
Analytics
Advanced AI
Advanced Verification
```

---

58. AI Development Rules

The AI layer must follow:

```
Input
 ↓
Prompt/Processing
 ↓
Structured Output
 ↓
Schema Validation
 ↓
Evidence Validation
 ↓
Business Rules
 ↓
Database
```

Never directly trust raw LLM output.

Never allow AI to:

- Invent documents
- Invent verification results
- Invent certificate numbers
- Invent evidence

- Claim unavailable external verification
- Bypass authorization

---

59. AI Provider Abstraction

Use:

```
AIProvider
```

with methods such as:

```
extract_requirements()
classify_document()
extract_entities()
match_evidence()
analyze_compliance()
```

Business logic should call:

```
AIService
```

instead of directly importing a specific AI SDK everywhere.

This allows providers to be changed later.

---

60. Verification Provider Abstraction

Use:

```
VerificationProvider
```

Implement:

```
MockVerificationProvider
```

first.

Later:

```
GSTVerificationProvider
CertificateVerificationProvider
OtherAuthorizedProvider
```

Each provider must follow the same interface.

---

61. Configuration

Centralize application settings.

Configuration categories:

```
Application
Database
Authentication
AI
OCR
Storage
Verification
CORS
File limits
Processing
Logging
```

Use environment variables.

---

62. Local Development

Recommended startup:

```
docker compose up -d postgres
```

Backend:

```
uvicorn app.main:app --reload
```

Frontend:

```
npm run dev
```

Worker:

```
worker startup command
```

Exact commands should be documented in README.

---

63. Docker

Create Dockerfiles for:

```
frontend
backend
worker
```

Docker Compose should provide development dependencies.

Example architecture:

```
frontend
backend
postgres
worker
storage
```

Keep production configuration separate from development configuration.

---

64. CI/CD

Set up CI pipeline for:

```
Install dependencies
 ↓
Lint
 ↓
Type checking
 ↓
Unit tests
 ↓
Integration tests
 ↓
Security checks
 ↓
Build
```

Do not deploy if critical tests fail.

---

65. Performance Optimization

Optimize only after correctness.

Important areas:

- Database indexes
- Pagination
- Async processing
- Background workers
- Document processing queues
- Caching where appropriate
- Efficient queries
- File streaming

Do not load large documents completely into memory unnecessarily.

---

66. Scalability

Initial architecture:

```
Frontend
 ↓
FastAPI
 ↓
PostgreSQL
 ↓
Worker
 ↓
Storage
```

Future:

```
Load Balancer
      ↓
Multiple API instances
      ↓
Queue
      ↓
Multiple workers
      ↓
PostgreSQL
```

```
      +
Object Storage
```

Keep API servers stateless wherever possible.

---

67. Development Milestones

Milestone 1

Application boots successfully.

Deliver:

- Frontend
- FastAPI
- PostgreSQL
- Docker
- Environment configuration

Milestone 2

Authentication + RBAC complete.

Milestone 3

Tender and bid management complete.

Milestone 4

Secure document management complete.

Milestone 5

OCR + extraction complete.

Milestone 6

Compliance engine complete.

Milestone 7

Verification + human review complete.

Milestone 8

Reports + dashboards complete.

Milestone 9

Security + testing complete.

Milestone 10

End-to-end SIH demo ready.

---

68. Final Development Flow

The agent should build the platform in this exact logical order:

```
1. Setup
   ↓
2. Database
   ↓
3. Authentication
   ↓
4. RBAC
   ↓
5. Tender Management
   ↓
6. Bid Management
   ↓
7. Secure File Management
   ↓
8. OCR
   ↓
9. Requirement Extraction
   ↓
10. Entity Extraction
   ↓
11. Evidence System
   ↓
12. Rule Engine
   ↓
13. AI Compliance
   ↓
14. Cross-Document Verification
   ↓
15. Verification Providers
   ↓
16. Human Review
```

```
   ↓
17. Reports
   ↓
18. Dashboards
   ↓
19. Audit System
   ↓
20. Security Hardening
   ↓
21. Automated Testing
   ↓
22. Docker Deployment
   ↓
23. End-to-End Demo
```

---

69. Critical Implementation Rules

The coding agent MUST follow these rules:

1. Do not put secrets in source code.
2. Do not bypass backend authorization.
3. Do not trust frontend roles.
4. Do not allow cross-bid document access.
5. Do not use raw SQL with unsafe string interpolation.
6. Do not execute uploaded files.
7. Do not treat AI output as verified truth.
8. Do not fabricate verification results.
9. Do not expose AI API keys to the frontend.
10. Do not store plaintext passwords.
11. Do not overwrite audit history.
12. Do not silently discard original evidence.
13. Do not automatically call mismatches fraud.
14. Use human review for uncertain/high-impact cases.
15. Preserve evidence for compliance decisions.
16. Keep AI/OCR/verification providers replaceable.
17. Use database migrations.
18. Write tests for critical business logic.
19. Keep API routes thin and services modular.
20. Make the application runnable locally before adding advanced features.

---

70. Final Acceptance Test

The development is successful only when this scenario works end-to-end:

```
Officer Login
      ↓
Create Tender
      ↓
Upload Tender PDF
      ↓
System Extracts Requirements
      ↓
Officer Reviews Requirements
      ↓
Bidder Login
      ↓
Submit Bid
      ↓
Upload Supporting Documents
      ↓
Secure File Validation
      ↓
OCR / Text Extraction
      ↓
Document Classification
      ↓
Entity Extraction
      ↓
Evidence Creation
      ↓
Requirement Matching
      ↓
Rule Engine
      +
AI Analysis
      +
Verification
      ↓
Compliance Results
      ↓
Uncertain Cases → Reviewer
      ↓
Reviewer Decision
      ↓
Final Compliance Status
      ↓
PDF Report
      ↓
Complete Audit Trail
```

The final platform must demonstrate that the system can move from a **raw tender document to an evidence-backed, explainable and auditable compliance decision** while maintaining strict security and role isolation.